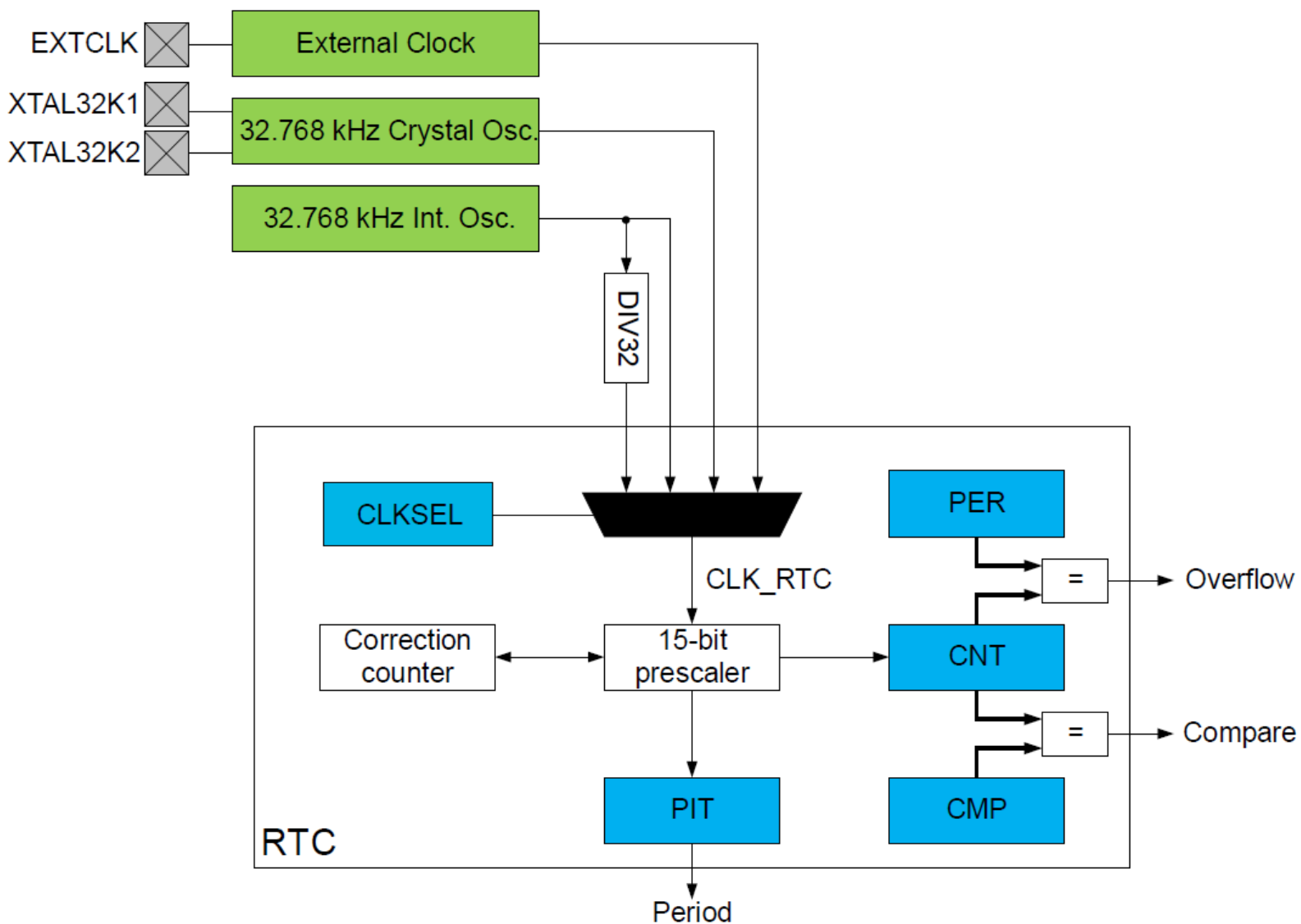


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 10 - Real-Time Counter (RTC) Basics



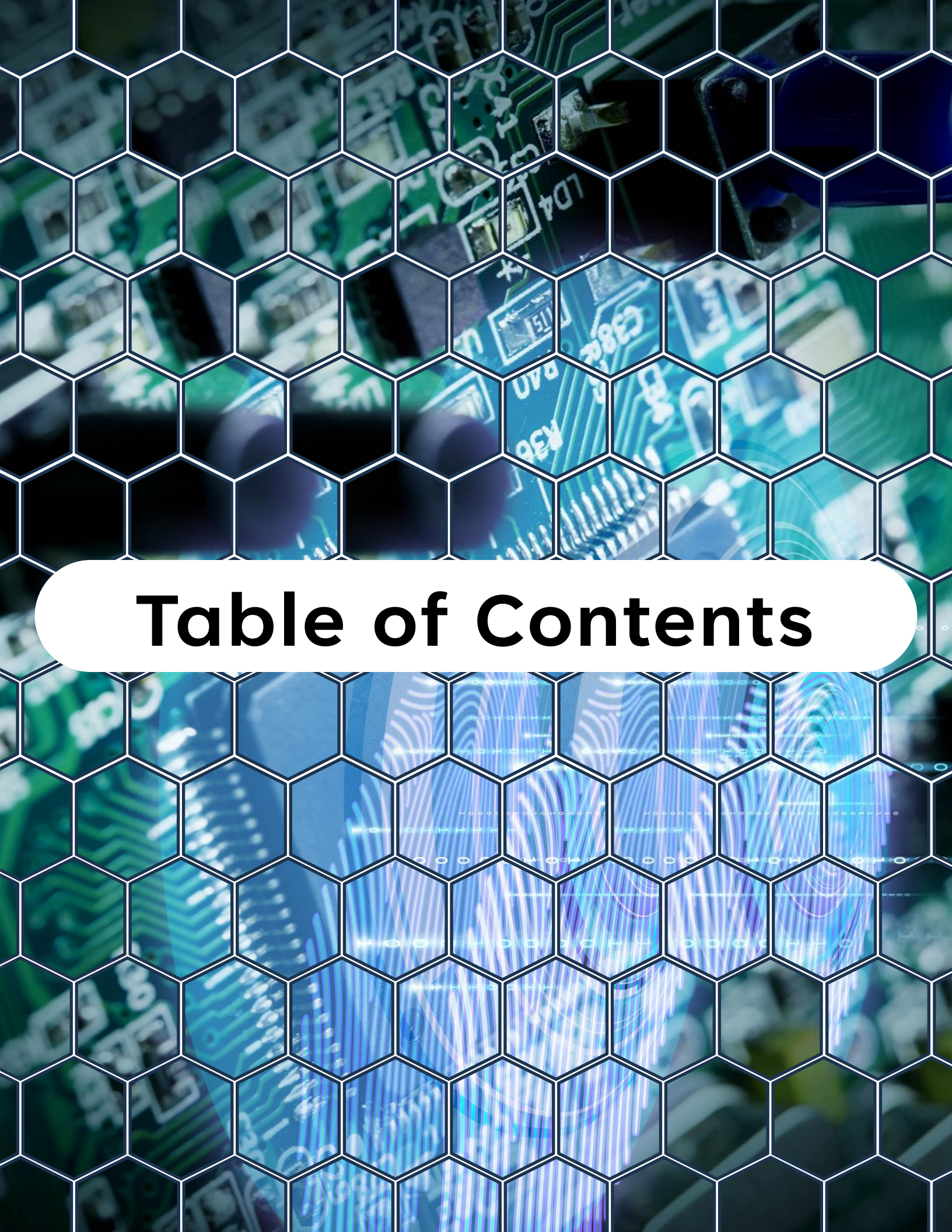
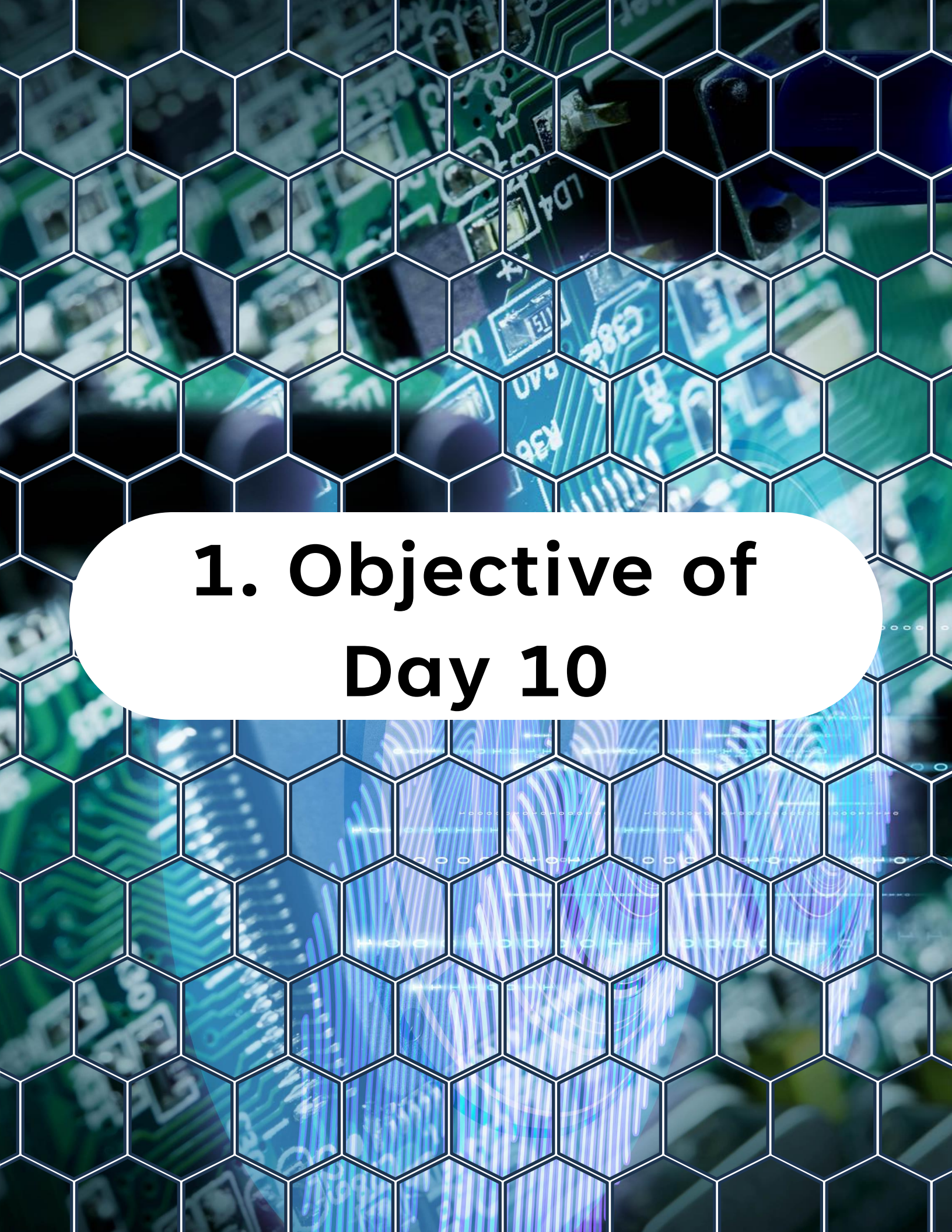


Table of Contents

Table of Contents

1. Objective of Day 10
2. Why RTC Exists
3. RTC Architecture in AVR128DA48
4. RTC Clock Domain
5. RTC vs TCB - When to Use Which
6. Basic RTC Periodic Interrupt Concept
7. Practical Lab - 1-Second Heartbeat Using RTC
8. Code Example - RTC 1 Hz Periodic Interrupt
9. Understanding RTC_PERIOD_CYC32768_gc
10. RTC While CPU Sleeps
11. Example - Sleep Until RTC Interrupt
12. Debugging RTC Issues
13. Common RTC Mistakes
14. What You Learned Today
15. What Comes Next



1. Objective of Day 10

1. Objective of Day 10

Today you move into a different timing world:
low-frequency, low-power, long-interval timing.

The **Real-Time Counter (RTC)** is not designed
for high-speed PWM or microsecond precision.

It is designed for:

- 1-second ticks
- Long periodic wakeups
- Timekeeping
- Low-power operation during sleep

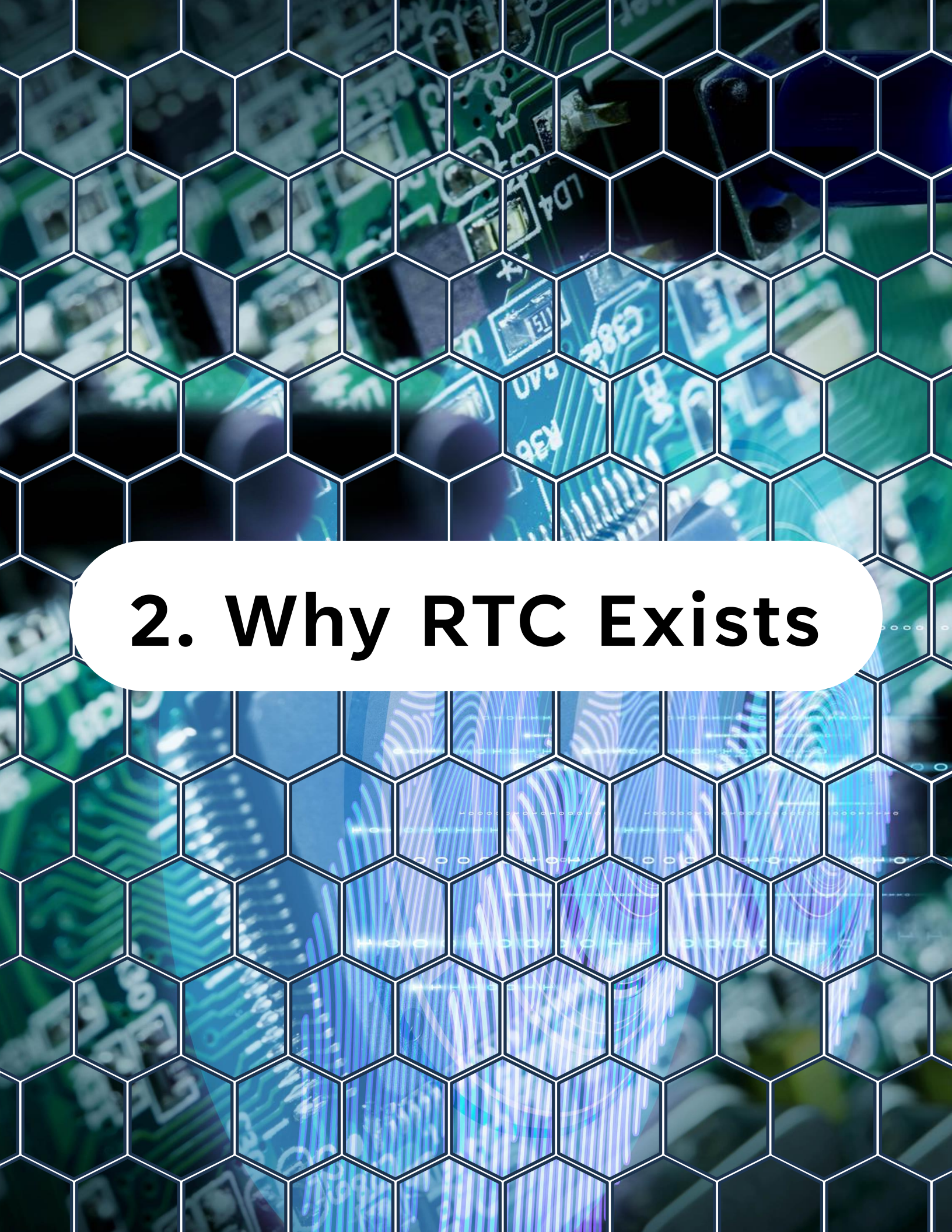
By the end of this day, you will:

- Understand how the **RTC** clock domain differs from **TCA/TCB**
- Configure **RTC** to generate periodic interrupts

1. Objective of Day 10

- Create a 1-second heartbeat
- Run **RTC** while the CPU is sleeping
- Understand when to use **RTC** vs **TCB**

This is where you start thinking about power-aware firmware.



2. Why RTC Exists

2. Why RTC Exists

You already have **TCA** and **TCB**. Why another timer?

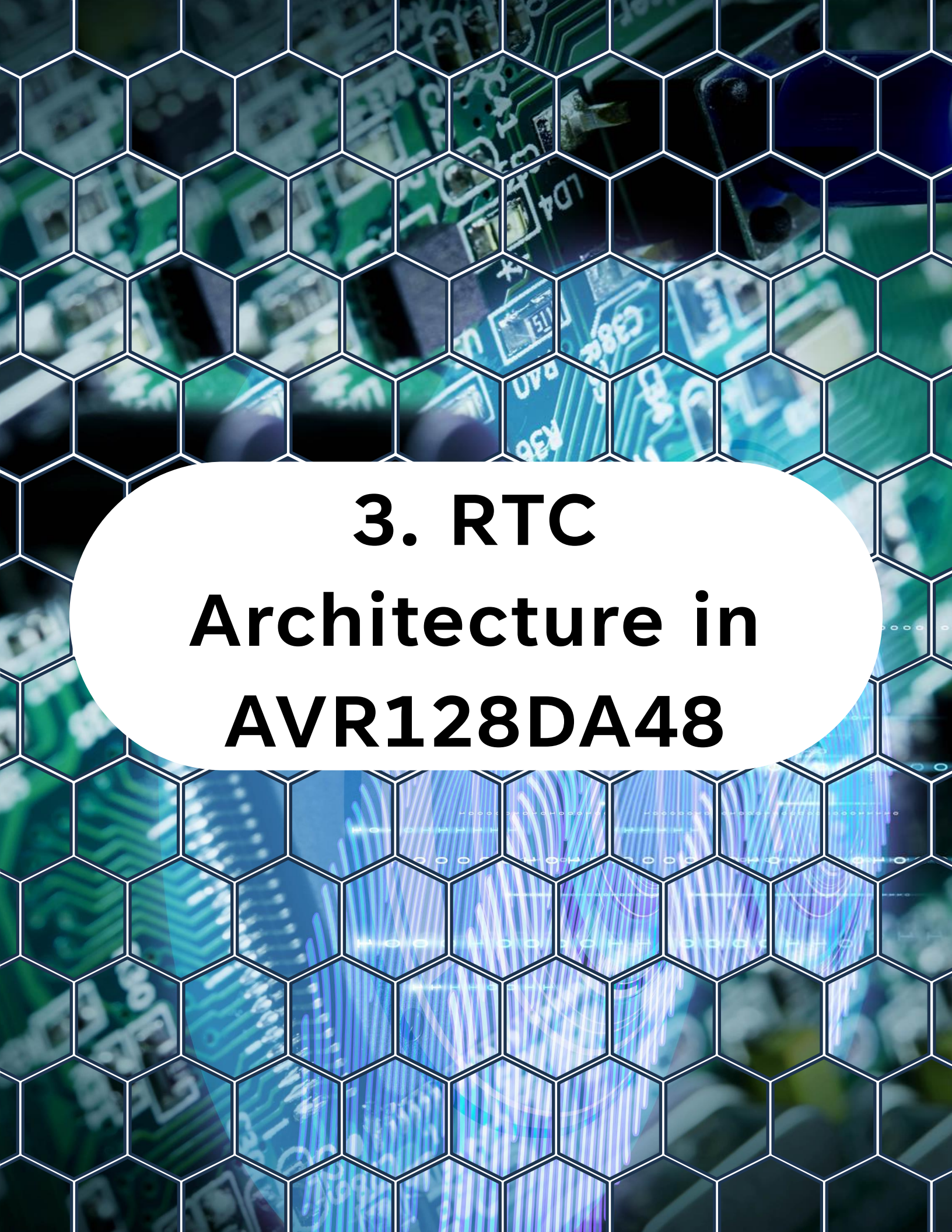
Because high-speed timers:

- Consume more power
- Depend on the main clock
- Stop or change behavior if the main clock changes

RTC:

- Runs from a low-frequency oscillator (typically 32.768 kHz)
- Lives in a separate clock domain
- Continues running in sleep modes
- Is ideal for long, stable time intervals

In real products, **RTC** is often the backbone of low-power scheduling.



3. RTC Architecture in AVR128DA48

3. RTC Architecture in AVR128DA48

On the **AVR128DA48**, the RTC:

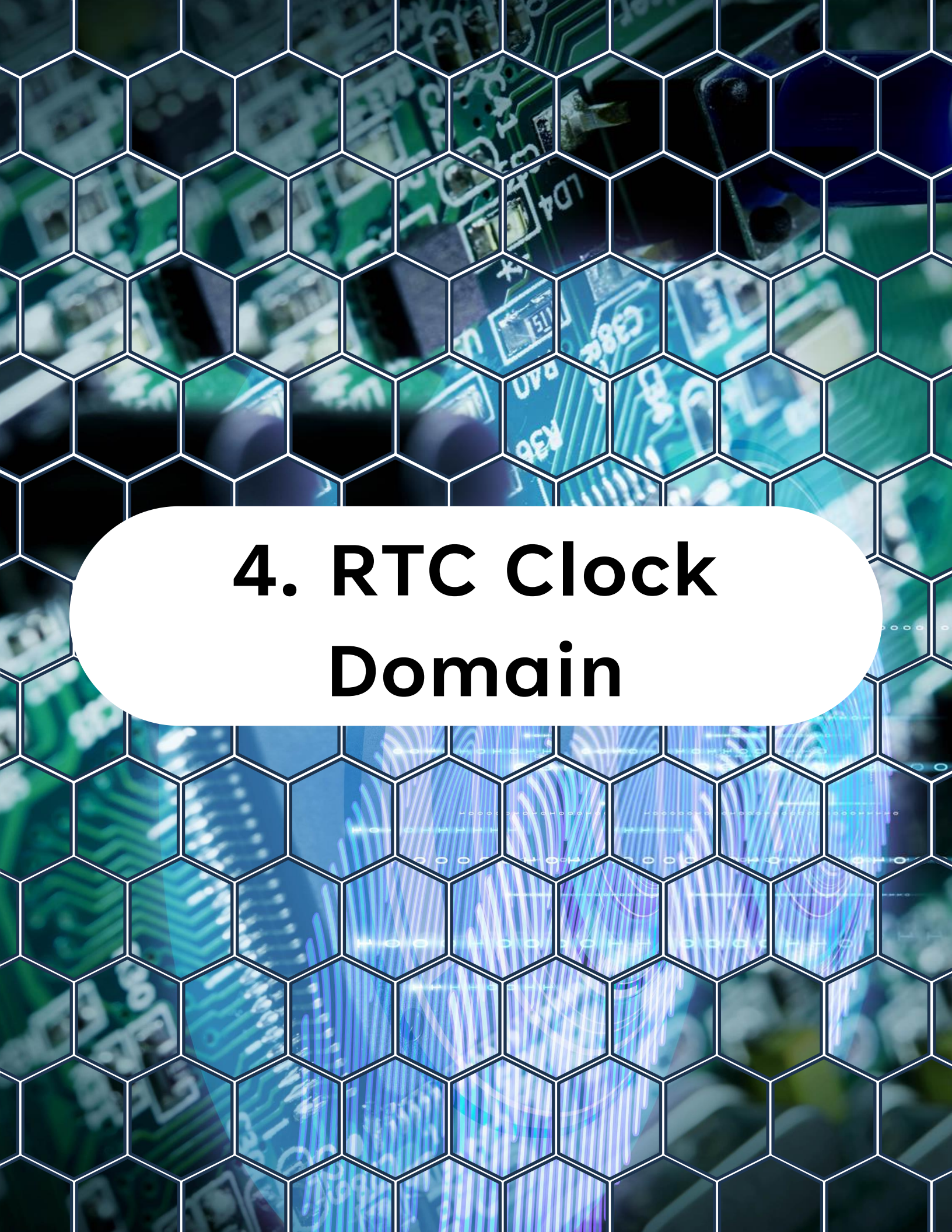
- Uses the internal 32 kHz oscillator (OSCULP32K or external crystal)
- Has a 16-bit counter
- Supports periodic interrupts
- Can generate overflow or compare events

The key idea is this:

RTC does not run at 24 MHz.

It runs at 32,768 Hz (or similar low-frequency source).

That changes your timing math completely.



4. RTC Clock Domain

4. RTC Clock Domain

The 32.768 kHz clock is chosen because:

- $32768 = 2^{15}$
- Dividing it down to 1 Hz is simple
- It is common for timekeeping crystals

Example:

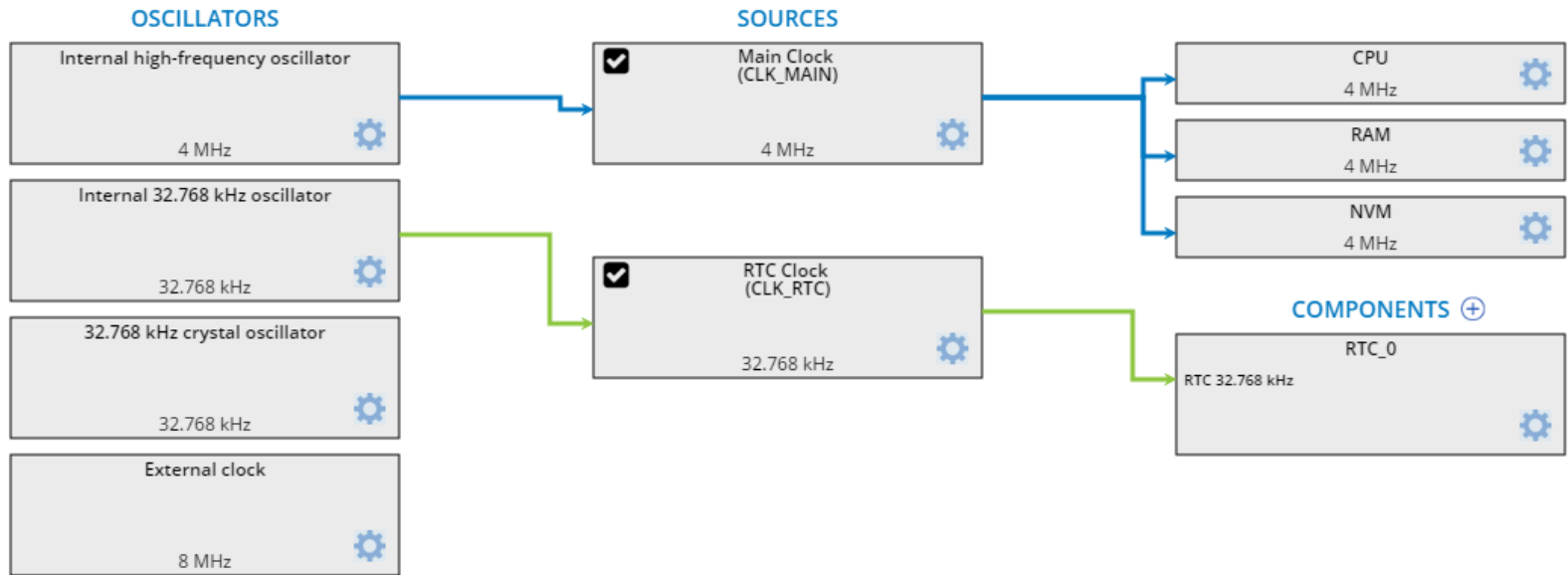
$$32768 \text{ Hz} / 32768 = 1 \text{ Hz}$$

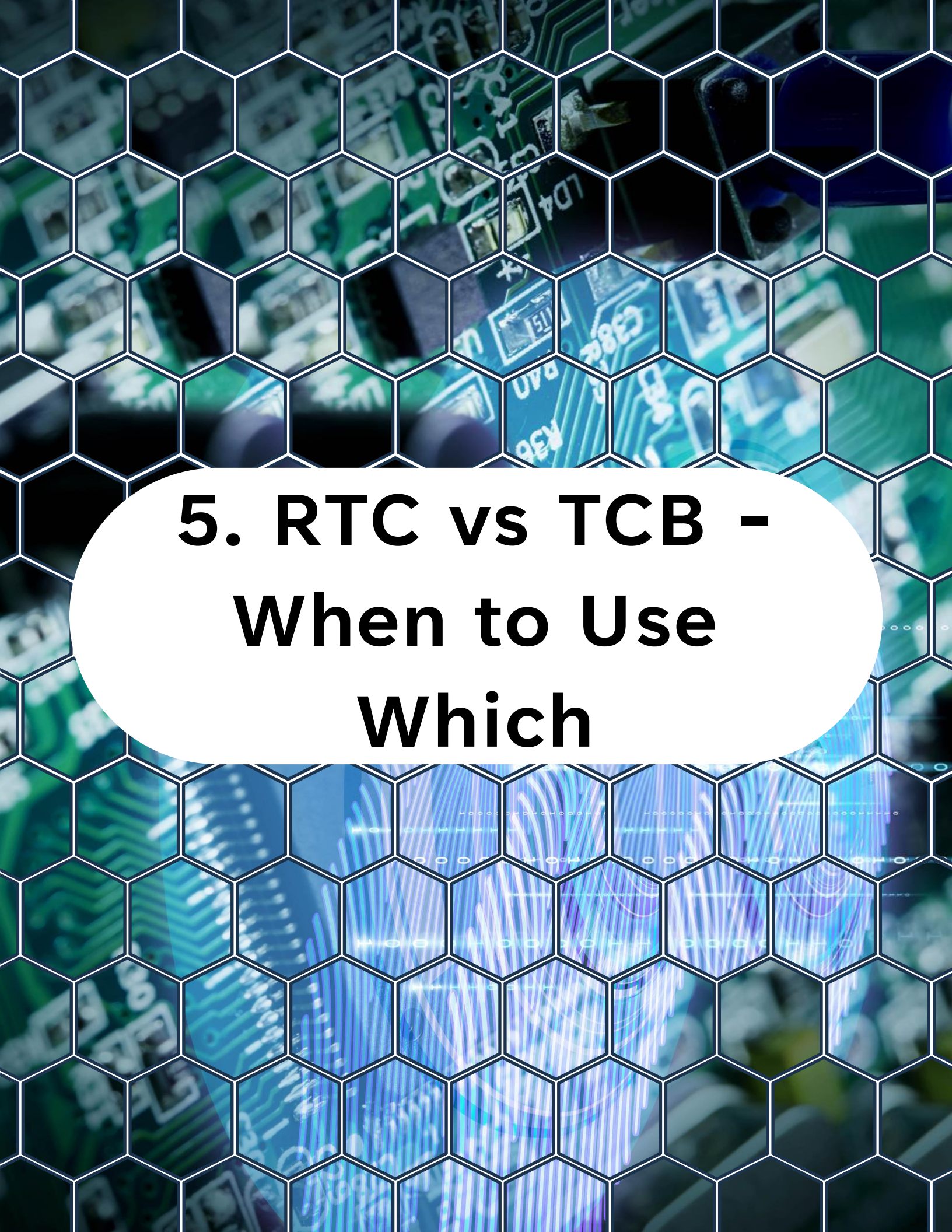
That means a 1-second interval is easy and exact.

Even when using the internal oscillator (not crystal-accurate), it is stable enough for most embedded tasks.

4. RTC Clock Domain

RTC Clock Domain





5. RTC vs TCB - When to Use Which

5. RTC vs TCB - When to Use Which

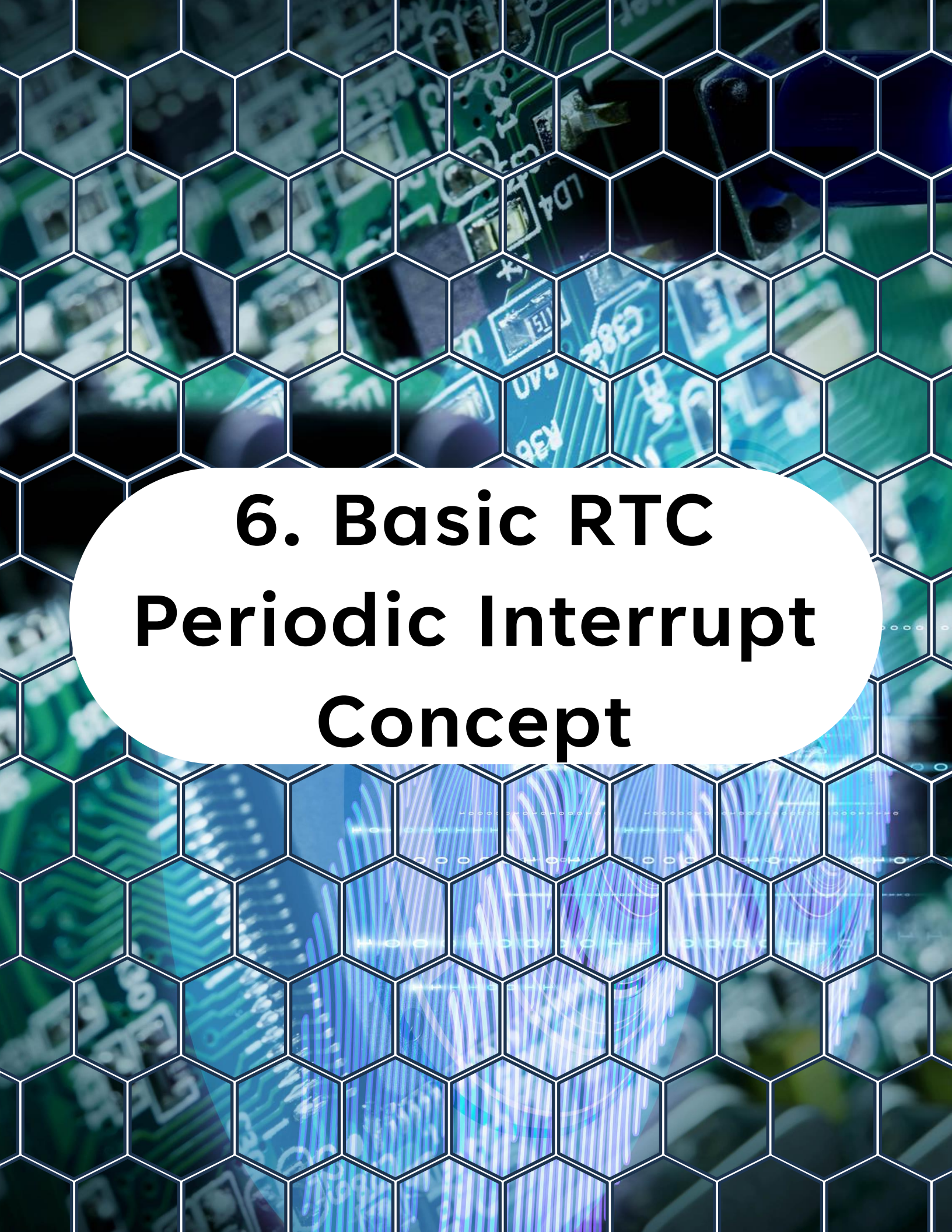
Use **TCB** when:

- You need millisecond resolution
- You need short, precise intervals
- CPU is running fast

Use **RTC** when:

- You need seconds or longer intervals
- You care about low power
- You want timer to run during sleep
- You are building timekeeping logic

This separation keeps your architecture clean.



6. Basic RTC Periodic Interrupt Concept

6. Basic RTC Periodic Interrupt Concept

RTC supports a periodic interrupt mode.

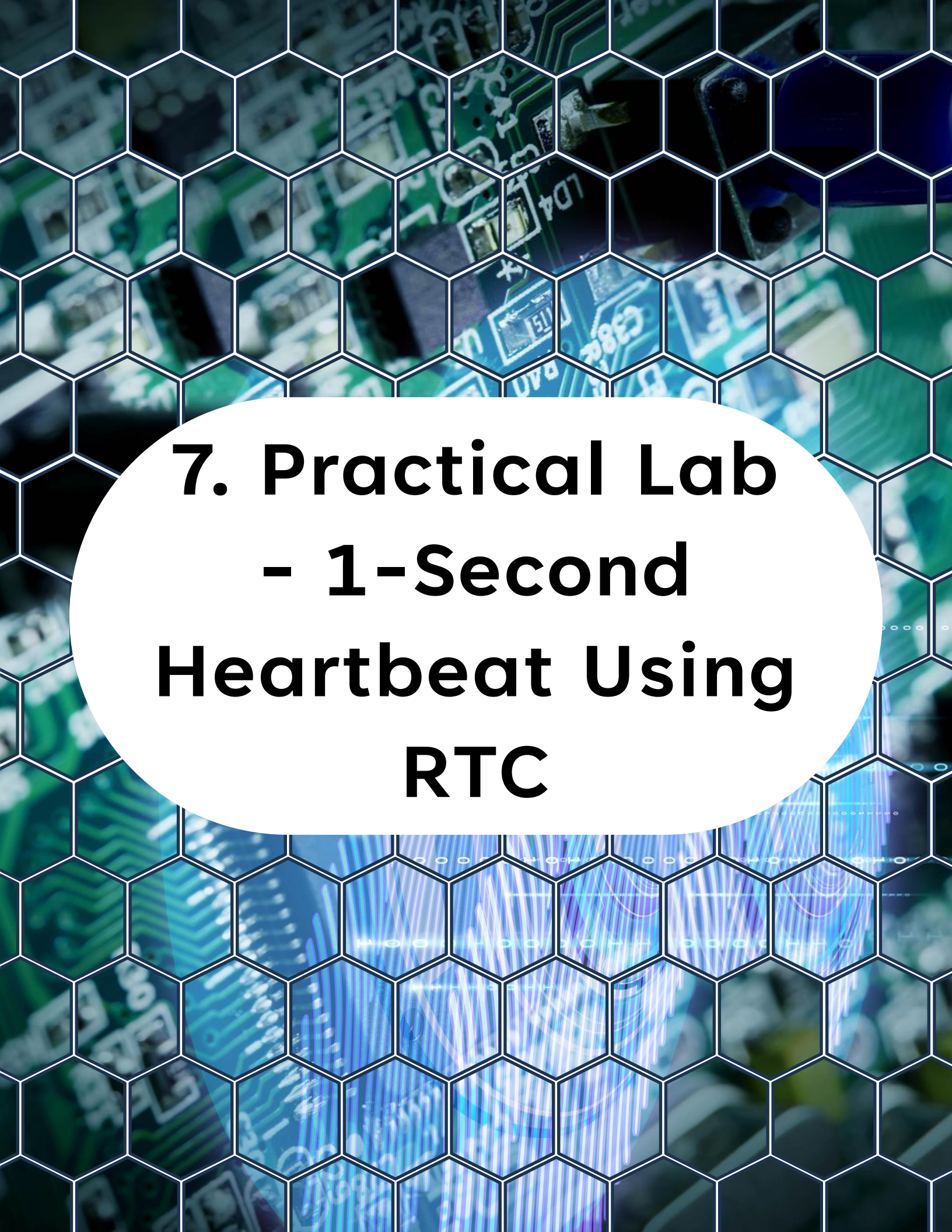
Instead of manually calculating compare values, you select:

- A clock source (32 kHz)
- A prescaler
- A periodic interrupt interval

The hardware then generates interrupts at fixed intervals like:

- 1 Hz
- 2 Hz
- 4 Hz
- etc.

This is very convenient.



7. Practical Lab - 1-Second Heartbeat Using RTC

7. Practical Lab - 1-Second Heartbeat Using RTC

Lab Goal

- Configure RTC to generate 1 interrupt per second
- Toggle an LED inside the ISR
- Verify 1-second timing with scope or visually

This proves:

- RTC clock domain works
- Interrupts work
- Timer runs independently from CPU speed

8. Code Example
- RTC 1 Hz
Periodic
Interrupt

8. Code Example - RTC 1 Hz Periodic Interrupt

This example assumes:

- 32.768 kHz internal oscillator
- Use periodic interrupt mode
- Toggle LED every second

```
1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3
4  static void gpio_led_init(void)
5  {
6      /* Example: LED on PA0 (adjust as needed) */
7      PORTA.DIRSET = PIN0_bm;
8      PORTA.OUTCLR = PIN0_bm;
9  }
10
11 static void rtc_init_1hz(void)
12 {
13     /* Ensure RTC is disabled before configuration */
14     RTC.CTRLA = 0;
15
16     /*
17      * Select 32 kHz internal oscillator
18      * Usually already enabled, but ensure it is stable if required.
19      */
20
21     /* Enable periodic interrupt at 1 Hz */
22     RTC.PITCTRLA = RTC_PERIOD_CYC32768_gc /* 32768 cycles = 1 second */
23                 | RTC_PITEN_bm;
24
25     /* Clear any pending interrupt flag */
26     RTC.PITINTFLAGS = RTC_PITINTFLAGS_bm;
```

8. Code Example - RTC 1 Hz Periodic Interrupt

```
24
25  /* Clear any pending interrupt flag */
26  RTC.PITINTFLAGS = RTC_PI_bm;
27
28  /* Enable periodic interrupt */
29  RTC.PITINTCTRL = RTC_PI_bm;
30 }
31
32 ISR(RTC_PIT_vect)
33 {
34     /* Clear interrupt flag */
35     RTC.PITINTFLAGS = RTC_PI_bm;
36
37     /* Toggle LED */
38     PORTA.OUTTGL = PIN0_bm;
39 }
```

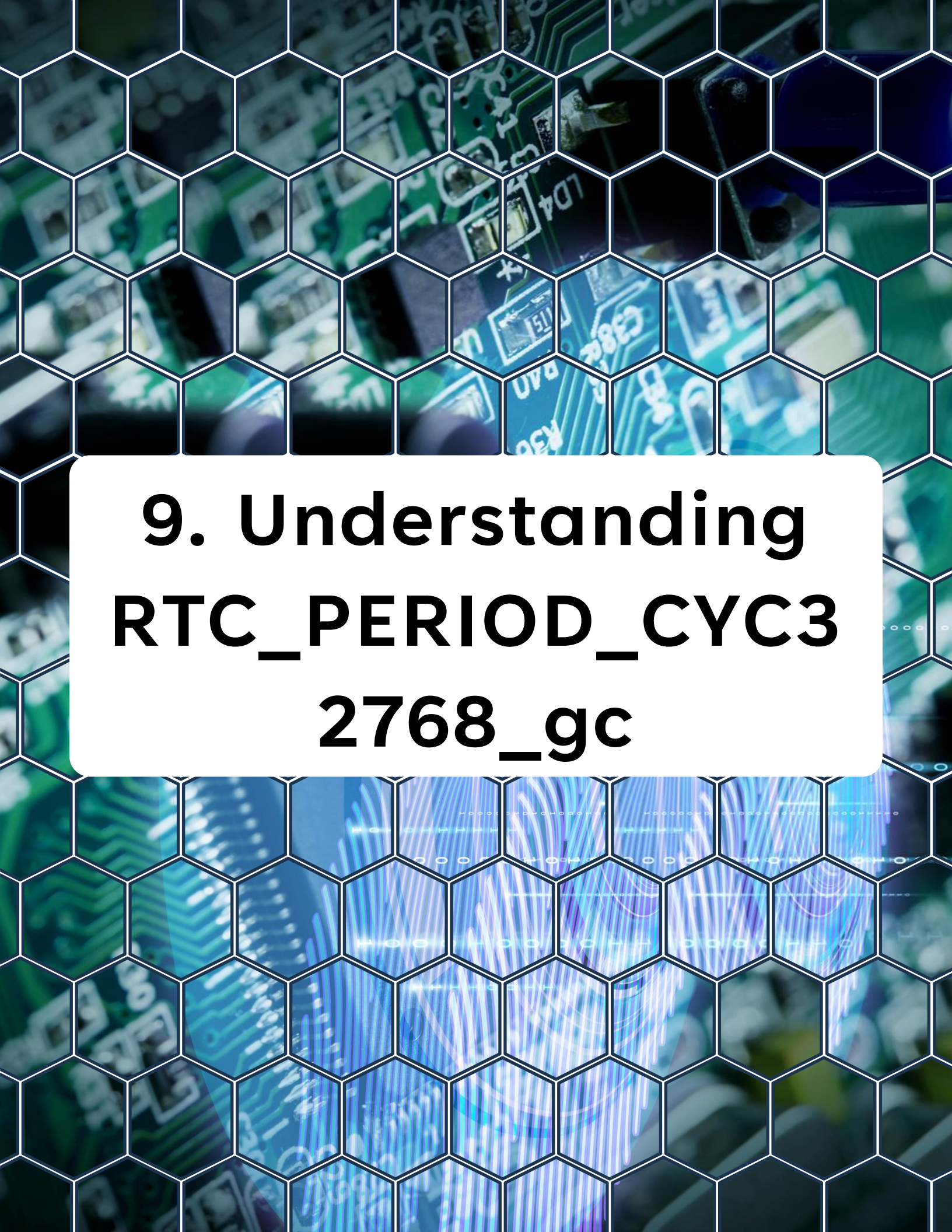
main()

```
1  int main(void)
2  {
3      gpio_led_init();
4      rtc_init_1hz();
5
6      sei();
7
8      while (1)
9      {
10         /* Main loop can do other work */
11     }
12 }
```

8. Code Example - RTC 1 Hz Periodic Interrupt

Expected result:

- LED toggles every 1 second
- Stable blinking independent of CPU load



9. Understanding RTC_PERIOD_CYC3 2768_gc

9. Understanding **RTC_PERIOD_CYC32768_gc**

The enum value:

RTC_PERIOD_CYC32768_gc

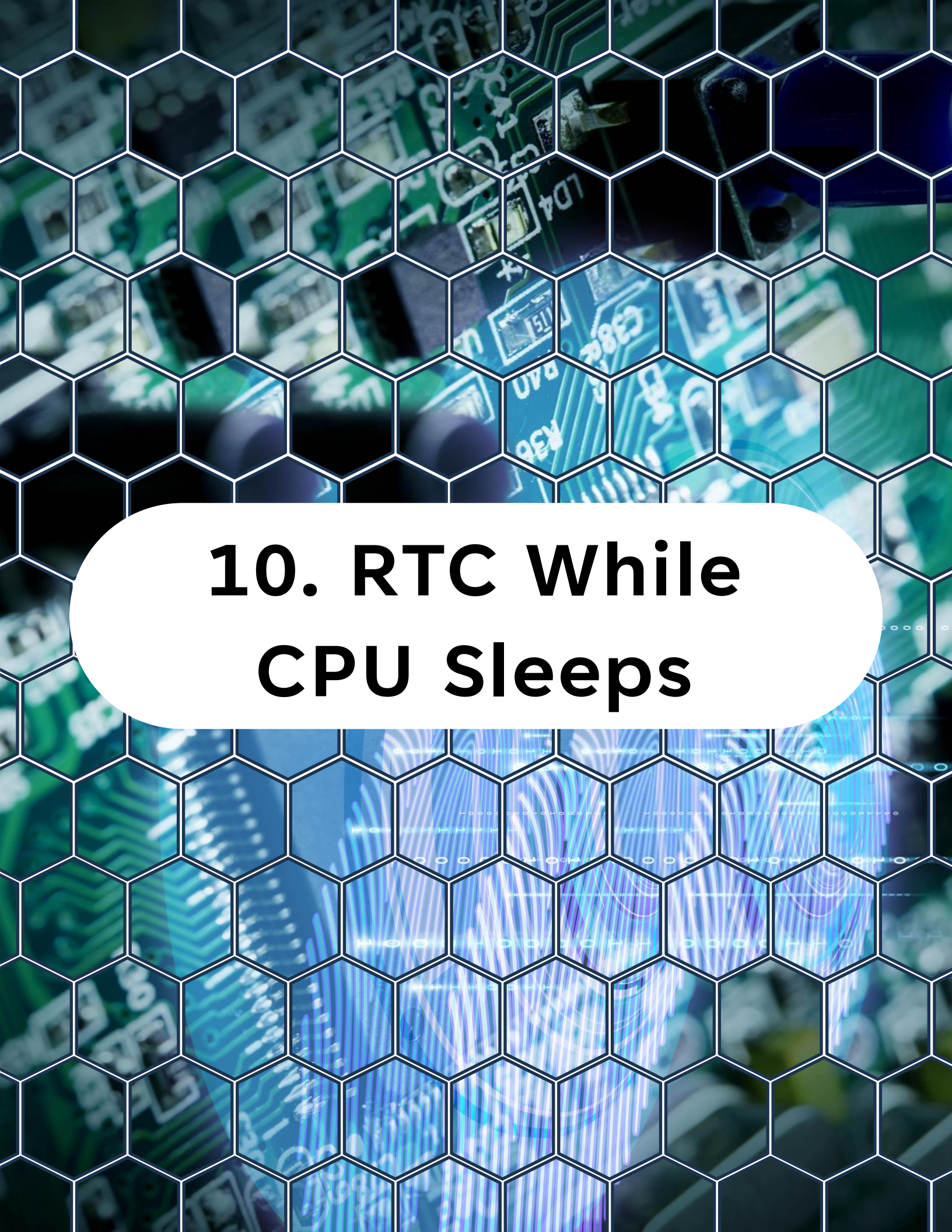
Means:

- Trigger interrupt every 32768 RTC clock cycles
- With 32,768 Hz clock -> 1 second

Other options include smaller intervals:

- 4096 cycles
- 8192 cycles
- etc.

Check the datasheet for exact enum values.



10. RTC While CPU Sleeps

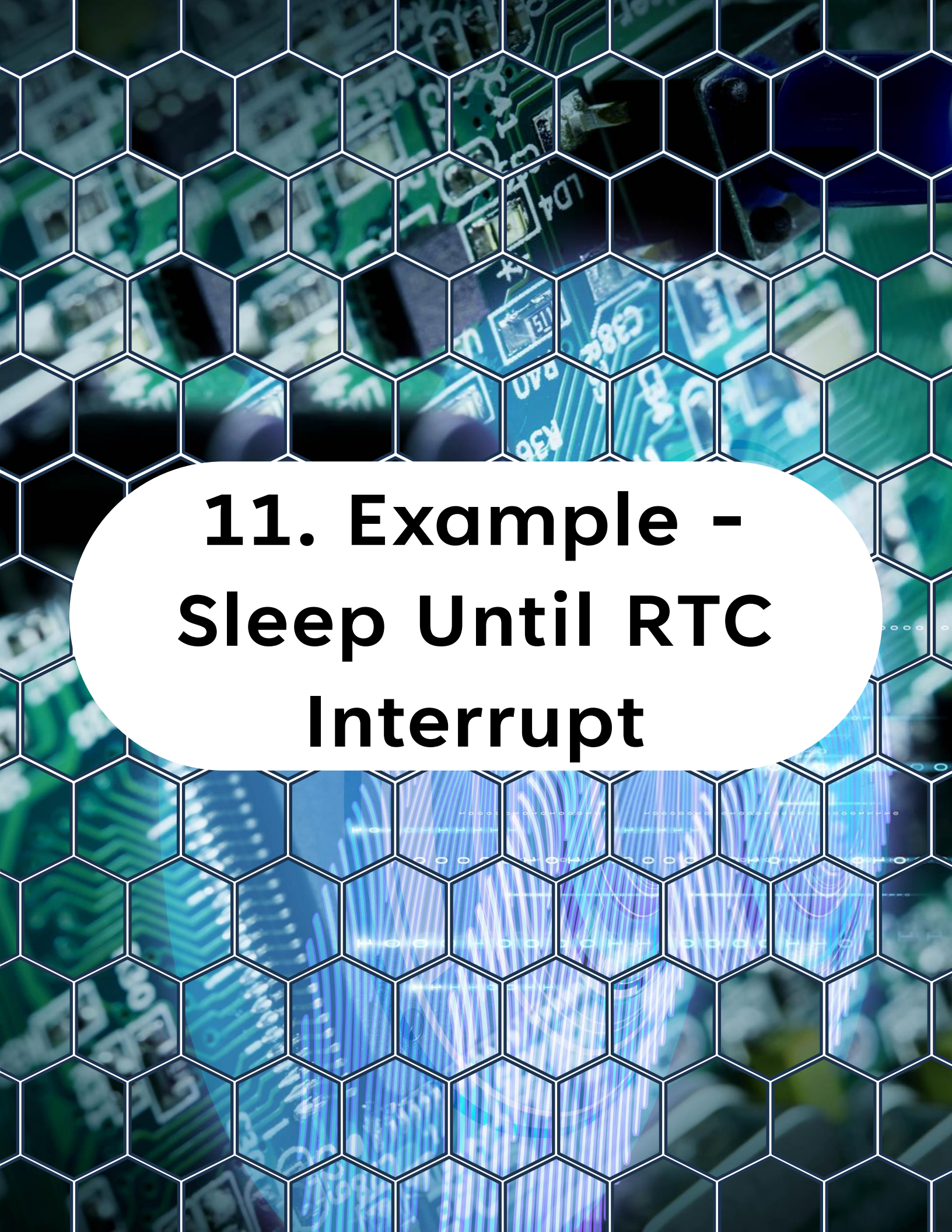
10. RTC While CPU Sleeps

Now we add the real advantage: sleep mode.

When the CPU enters sleep:

- Main clock stops (depending on sleep mode)
- RTC continues running
- RTC interrupt wakes CPU

This is the foundation of low-power embedded systems.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', 'U7A', and 'K30'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white oval contains the title text in a bold, black, sans-serif font.

11. Example - Sleep Until RTC Interrupt

11. Example - Sleep Until RTC Interrupt

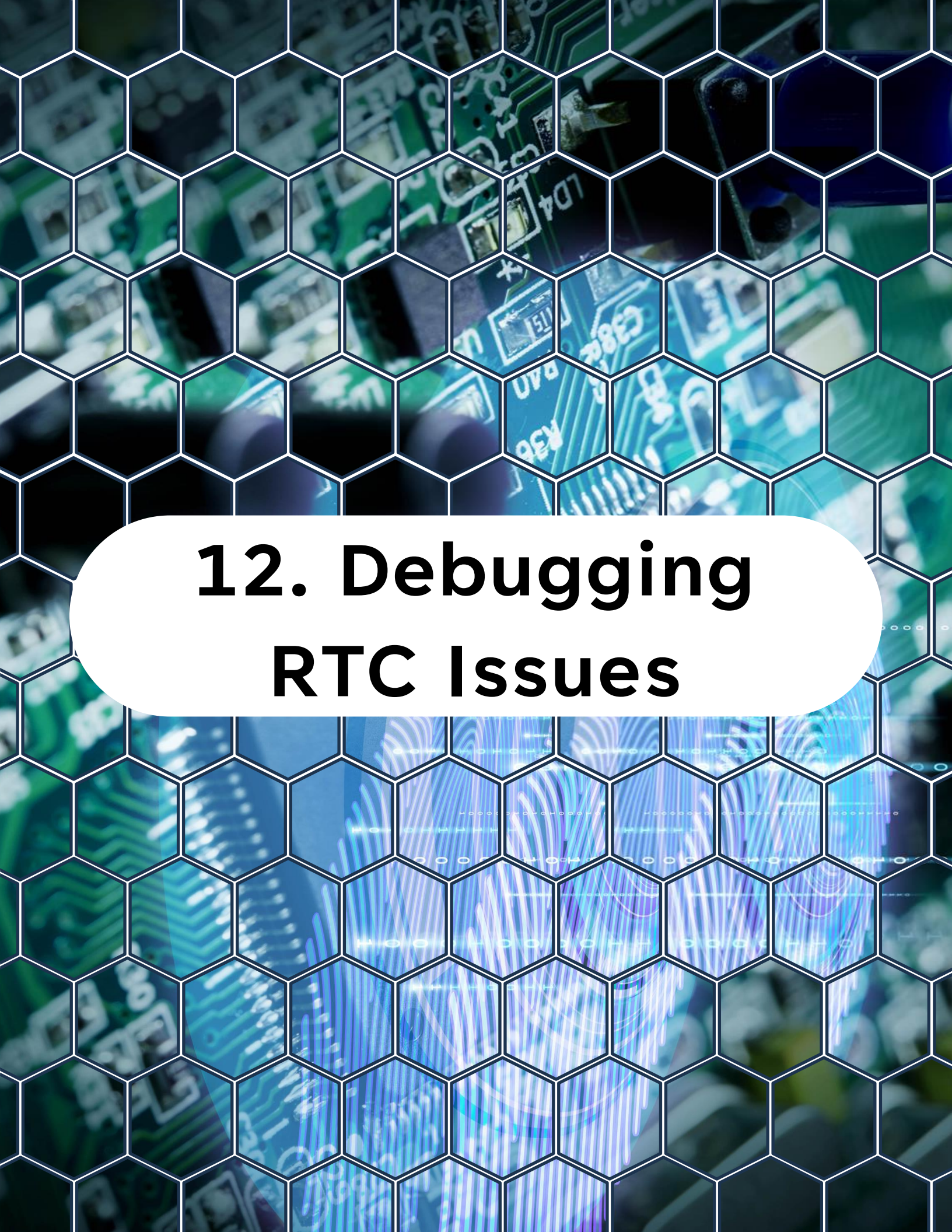
Add sleep to the previous example.

```
1  #include <avr/sleep.h>
2
3  int main(void)
4  {
5      gpio_led_init();
6      rtc_init_1hz();
7
8      sei();
9
10     set_sleep_mode(SLEEP_MODE_IDLE);
11
12     while (1)
13     {
14         sleep_mode();
15         /* CPU wakes up here after RTC interrupt */
16     }
17 }
```

Behavior:

- CPU sleeps
- RTC fires every second
- ISR toggles LED
- CPU returns to sleep

This is how battery-powered systems operate.



12. Debugging RTC Issues

12. Debugging RTC Issues

If LED does not blink:

- Verify global interrupts enabled
- Confirm **RTC.PITEN** bit is set
- Confirm **PIT** interrupt enabled
- Confirm interrupt flag cleared in ISR
- Confirm LED pin correct
- Confirm oscillator running (rare issue but possible)

If timing is off:

- Confirm which oscillator is used
- Confirm correct PERIOD enum value



13. Common RTC Mistakes

13. Common RTC Mistakes

- Forgetting to clear interrupt flag
- Assuming **RTC runs at 24 MHz**
- Using wrong enum value
- Forgetting that **RTC is 16-bit**
- Not enabling sleep correctly

RTC bugs are usually configuration mistakes, not logic mistakes.



14. What You Learned Today

14. What You Learned Today

By the end of Day 10, you now understand:

- The purpose of the Real-Time Counter
- How RTC differs from TCA and TCB
- How to generate a 1-second periodic interrupt
- How to use RTC in low-power sleep scenarios
- How RTC forms the backbone of long-interval scheduling

You now have:

- High-speed timing (TCA)
- Millisecond tick (TCB)
- Second-level low-power timing (RTC)

This is a complete timing toolbox.



15. What Comes Next

15. What Comes Next

Week 3 - Communication Peripherals

Starting with:

Day 11 - USART Fundamentals

You will:

- Configure asynchronous serial communication
- Calculate baud rates correctly
- Transmit and receive data without libraries
- Build your first real communication driver

From this point forward, your MCU will start talking to the outside world.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>